ACADEMIC PROJECT DOCUMENTATION | GROUP PROJECT

Student Enrollment & Course Management System

(SECMS)

Phase 2

Backend Development & Configurations

Salesforce Developer Edition

PROJECT TYPE	Group Project
PLATFORM	Salesforce Developer Edition
PHASE	Phase 2 – Backend Development & Configurations
TEAM LEAD	**AKASH C**
TEAM MEMBERS	A MOHAMMED IMRAN | SARANRAJ S
DOCUMENT STATUS	Draft v1.0 – based on Phase 1 requirements and project screenshots

Table of Contents

1. Cover Page . . . 1

About This Document: Evidence Key and Scope . . . 4

Scope of this document . . . 4

2. Phase 2 Overview . . . 5

2.1 Purpose of Backend Development and Configuration . . . 5

2.2 Phase 2 Objectives . . . 5

2.3 Traceability to Phase 1 Requirements . . . 5

2.4 Completed and Planned Work at a Glance . . . 5

3. Salesforce Org Setup . . . 6

3.1 Preparing the Salesforce Developer Org . . . 6

3.2 Login Process . . . 6

3.3 Setup Menu and Object Manager . . . 6

3.4 Configuration Areas Used in Phase 2 . . . 7

4. Creating and Configuring Objects . . . 8

4.1 Standard and Custom Objects . . . 8

4.2 Creating the Student Object . . . 8

4.3 Creating the Course, Instructor and Enrollment Objects . . . 9

5. Creating Fields . . . 10

5.1 Student Object (Student__c) . . . 10

5.2 Course Object (Course__c) . . . 10

5.3 Instructor Object (Instructor__c) . . . 11

5.4 Enrollment Object (Enrollment__c) . . . 11

6. Configuring Relationships . . . 12

6.1 Lookup Relationships and the Junction Object . . . 12

6.2 Relationship Summary . . . 12

6.3 Creating the Lookup Relationships . . . 13

6.4 How the Relationships Support Data Integrity . . . 14

7. Validation Rules . . . 15

7.1 How Validation Rules Maintain Accurate Data . . . 15

7.2 Rules Configured in the Salesforce Org . . . 15

7.3 Proposed Additional Rules (Not Yet Configured) . . . 16

7.4 Suggested Refinement of VR-1 . . . 16

7.5 Duplicate Enrollment Control (Proposed) . . . 17

8. Record-Triggered Flows . . . 17

8.1 Backend Automation Overview . . . 17

8.2 Enrollment Record Processing . . . 17

8.3 Enrollment Status Updates and Approval-Related Automation . . . 18
8.4 Instructor Assignment Based on Course Category . . . 21
8.5 Follow-Up Task Creation . . . 23
9. Backend Business Logic . . . 25
9.1 End-to-End Enrollment Processing Logic . . . 25
9.2 How the Components Work Together . . . 26
10. Testing and Verification . . . 26
10.1 Creating Records . . . 26
10.2 Required Field Validation . . . 26
10.3 Validation Rule Behaviour . . . 27
10.4 Relationship Verification . . . 27
10.5 Instructor Assignment . . . 27
10.6 Enrollment Status Changes and Approval . . . 28
10.7 Duplicate or Invalid Data . . . 28
11. Phase 2 Deliverables . . . 28
11.1 Deliverables and Their Status . . . 29
11.2 Items Requiring Confirmation Before Submission . . . 29
12. Phase 2 Conclusion . . . 30
Preparing for Phase 3: UI/UX Development & Customization . . . 30
References . . . 30

About This Document: Evidence Key and Scope

This document records the Phase 2 (Backend Development & Configurations) work for the **Student Enrollment & Course Management System (SECMS)**, built on a Salesforce Developer Edition org. It is written from two sources only: the **confirmed Phase 1 project document** (requirements, milestones and the Salesforce screenshots it contains) and the Phase 2 documentation brief supplied by the team.

To keep the report honest, every configuration item carries a status tag. Nothing is described as completed unless the screenshots or Phase 1 text support it.

Tag	Meaning
Confirmed	Supported by Phase 1 text and/or a project screenshot of the Salesforce org.
API to confirm	The field or object exists in the design, but its exact API name is not visible in the evidence. The name shown is a naming-convention proposal and must be checked in Setup.
To confirm	The evidence is unclear or inconsistent (for example, two different names for the same item). The team must verify it in the org before submission.
Proposed	A recommended design item that is not evidenced as built. It needs team approval before it is configured.
Pending	A test or verification step that has not yet been performed. Actual results are intentionally left blank.

Scope of this document

- **Included (Phase 2):** Salesforce org setup, custom objects, fields, relationships, validation rules, record-triggered flows, the enrollment approval process, backend business logic, test cases and deliverables.
- **Not documented here:** Phase 3 (UI/UX: tabs, Lightning app, page layouts, record types), Phase 4 and Phase 5 work. Screenshots that belong to those phases have been left out on purpose.
- **Privacy:** Screenshots showing usernames, e-mail addresses of real accounts or verification links from the Phase 1 file have been excluded from this report.

Note: the sprint dates in the Phase 1 tracker (October and November 2022) belong to the course template and are not used as implementation dates in this document.

2. Phase 2 Overview

2.1 Purpose of Backend Development and Configuration

In a Salesforce application, the **backend** is everything that stores and protects data and runs business logic without user effort: objects (database tables), fields (columns), relationships (links between tables), validation rules, automation flows and approval processes. All of this is built declaratively (with clicks, not code) in Setup.

For SECMS, Phase 2 turns the requirements approved in Phase 1 into a working data and automation layer. A student record, a course record and an instructor record are stored once; an **Enrollment** record links them and drives the approval and instructor assignment logic. Without this layer, the screens built in Phase 3 would have nothing reliable to display or update.

2.2 Phase 2 Objectives

- Prepare and access the Salesforce Developer Org used for the project.
- Create the four custom objects: Student, Course, Instructor and Enrollment.
- Create the fields needed to capture student details, course information and enrollment status.
- Link the objects with lookup relationships so that every enrollment belongs to a student, a course and an instructor.
- Protect data quality with validation rules (institutional e-mail domain, fees paid before approval).
- Automate enrollment processing with record-triggered flows and an approval process.
- Prepare test cases so that the backend can be verified before the user interface work of Phase 3 begins.

2.3 Traceability to Phase 1 Requirements

Phase 1 requirement	Phase 2 backend component	Section
FR-1 Student Management: manage student records	Student__c object, student fields, e-mail domain validation rule	4, 5, 7
FR-2 Course Management: create and manage courses	Course__c object, Category and Course Fee fields	4, 5
FR-3 Enrollment Processing: request, approve, reject enrollments	Enrollment__c object, lookups, Fees Paid validation rule, approval process, enrollment flows	4, 6, 7, 8
FR-4 Instructor Assignment: auto-assign instructors	AutoAssign_Instructor flow (Technical / Language / Non-Technical rules)	8
FR-5 Reporting: dashboards and reports	Not part of Phase 2. The Phase 2 data model supplies the data that reports will use later.	—
NFR-3 Reliability: no duplicate registration, accurate status updates	Required lookups, delete restrictions and validation rules (duplicate control is proposed, see 7.5)	6, 7

2.4 Completed and Planned Work at a Glance

The table separates what the project evidence shows as done from what is still planned or unverified.

Work item	Evidence in the project file	Status
Developer Org creation and first login	Sign-up steps and Setup Home screenshot	Completed (evidenced)
Four custom objects	Student object screenshots; Course, Instructor and Enrollment appear in lookup, validation and flow screenshots	Completed (evidenced)
Fields	Phase 1 field tables; several fields visible in screenshots; some API names not visible	Partly evidenced; API names to confirm
Three lookup relationships	Screenshots for Student, Course and Instructor lookups	Completed (evidenced)
Two validation rules	Rule definition screenshots (formula and error message)	Completed (evidenced); saved names to confirm
Four record-triggered flows	Flow element screenshots; Flow Builder shows AutoAssign Instructor as Active (V18)	Built (evidenced); activation shown for one flow only

Work item	Evidence in the project file	Status
Approval process	Name, entry criteria, approval step, field updates and e-mail alert screenshots	Configured (evidenced); activation not shown
Test execution and verification evidence	None yet	Planned

3. Salesforce Org Setup

3.1 Preparing the Salesforce Developer Org

A Salesforce Developer Edition org is a free, full-featured environment for building and testing applications. The team created its org through the official sign-up page listed in the Phase 1 document (Milestone 1, Activity 1).

Step	Action	Details from Phase 1
1	Open the sign-up page	developer.salesforce.com/signup
2	Complete the sign-up form	First name and last name, e-mail address, Role = Developer, Company = college name, Country = India, postal code, and a username in e-mail format (for example username@organization.com; it need not be a real mailbox)
3	Submit	Click **Sign me up**
4	Activate the account	Open the verification e-mail (it can take 5–10 minutes) and click **Verify Account**
5	Set credentials	Enter a new password (minimum 8 characters, 1 letter, 1 number), choose a security question and answer, then click **Change Password**
6	First login	Salesforce redirects to the **Setup Home** page

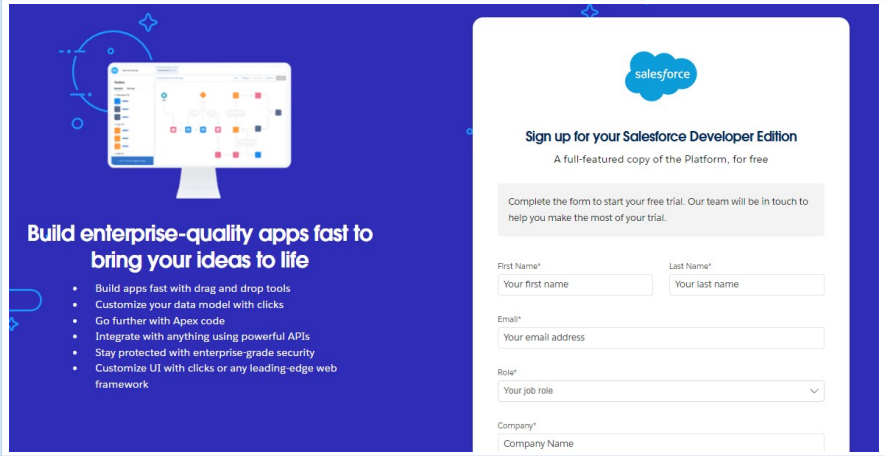


Figure 1: Salesforce Developer Edition sign-up form (Source: project screenshot)

3.2 Login Process

After activation, the team signs in using the org username and the password set in step 5. The verification e-mail also provides a personal *My Domain* login URL that should be saved for later access. For security, the org username, password and login URL are not reproduced in this report.

3.3 Setup Menu and Object Manager

Setup is the administration area of Salesforce, opened from the gear icon at the top right of the screen. It has a **Quick Find** box on the left that filters the menu, and a **Setup Home** page as the landing screen. Next to Home sits the **Object Manager** tab, which lists every standard and custom object and gives access to each object's fields, relationships, validation rules and other settings.

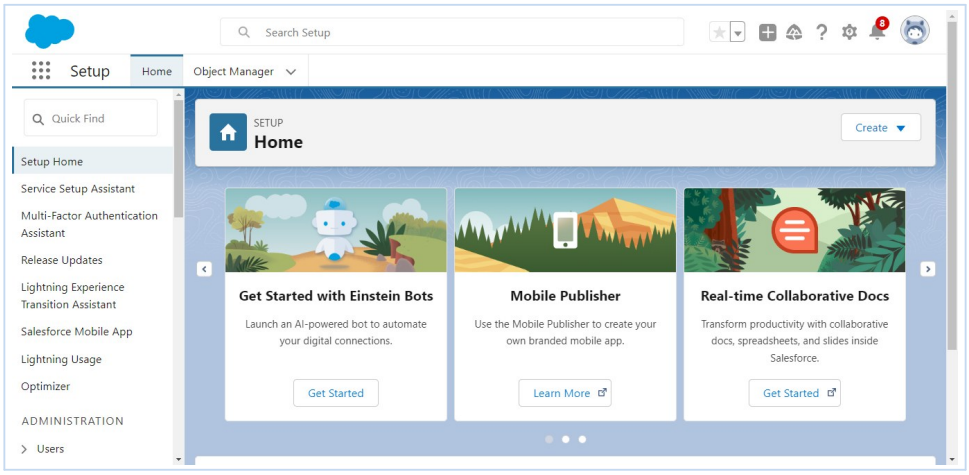


Figure 2: Setup Home page with the Object Manager tab and the Quick Find box (Source: project screenshot)

3.4 Configuration Areas Used in Phase 2

Setup area	How to reach it	Used in Phase 2 for
Object Manager	Setup > Object Manager	Creating objects; fields and relationships; validation rules on each object
Fields & Relationships	Object Manager > object > Fields & Relationships	Creating fields and the three lookup relationships on Enrollment
Validation Rules	Object Manager > object > Validation Rules	Student e-mail domain rule and Fees Paid rule
Flows	Setup > Flows (Flow Builder)	Record-triggered flows for enrollment automation
Approval Processes	Setup > Approval Processes	Enrollment approval by the Training Manager
Field Updates and Email Alerts	Created from the approval process actions	Setting the status to Approved or Rejected and notifying the student

Other Setup areas seen in the Phase 1 file (tabs, App Manager, page layouts, record types, profiles, permission sets and sharing settings) are part of later phases and are not documented here.

4. Creating and Configuring Objects

4.1 Standard and Custom Objects

Salesforce provides **standard objects** (such as Account, Contact and Task) that exist in every org. When a project needs its own data structure, the administrator creates **custom objects**; their API names always end in **__c**. According to the approved design, all four SECMS data objects are custom objects.

Object label	Plural label	API name	Type	Purpose	Status
Student	Students	Student__c	Custom	Stores student personal details, contact information and status	**Confirmed**
Course	Courses	Course__c	Custom	Stores course name, category, duration, fee and description	**Confirmed**
Instructor	Instructors	Instructor__c	Custom	Stores instructor name, code, expertise and contact details	**Confirmed**
Enrollment	Enrollments	Enrollment__c	Custom	Records each student's enrollment in a course; junction between Student and Course; also links the assigned Instructor	**Confirmed** **To confirm**

To confirm: in several screenshots the Enrollment record name is spelled "Enrolement Name" and a flow variable is displayed as "Triggering Enrolement__c", while the Phase 1 steps use *Enrollment__c*. The team must open Object Manager and confirm the exact object label, record-name label and API name, then use one spelling throughout.

Standard objects used by the backend

Standard object	Use in Phase 2
User	The flow Create_Followup_Task looks up the Admin user; approval steps and e-mail alerts refer to users
Task	The follow-up flow creates a Task record for the Admin when an enrollment is requested

4.2 Creating the Student Object

The Student object is the model for the other three objects. It was created from Setup > Object Manager > Create > Custom Object with the settings below (Phase 1, Milestone 2).

Setting	Value used	Status
Label / Plural Label	Student / Students	**Confirmed**
Object Name (API name Student__c)	Student	**Confirmed**
Record Name and Data Type	Student Name, Text	**Confirmed**
Optional features	Allow Reports, Allow Activities and Track Field History are ticked; Allow in Chatter Groups is not ticked	**Confirmed**
Object classification	Allow Sharing, Allow Bulk API Access and Allow Streaming API Access are ticked (defaults)	**Confirmed**
Deployment Status	Deployed	**Confirmed**
Search Status	Allow Search ticked, then **Save**	**Confirmed**

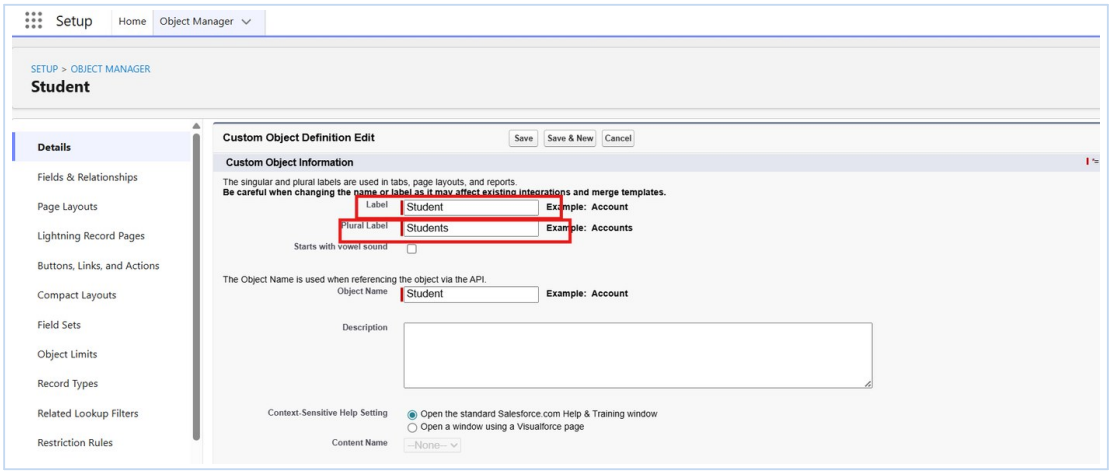


Figure 3: Custom Object Definition Edit: label, plural label and object name (Source: project screenshot)

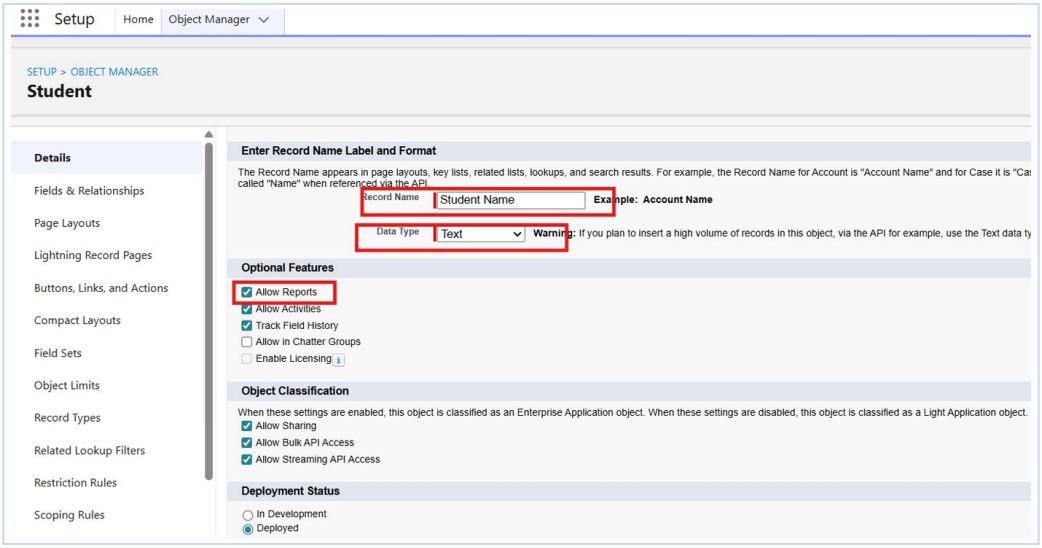


Figure 4: Record name, data type, optional features and deployment status (Source: project screenshot)

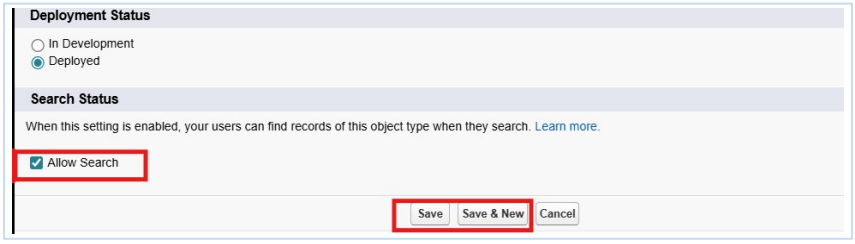


Figure 5: Search status and saving the object (Source: project screenshot)

4.3 Creating the Course, Instructor and Enrollment Objects

Phase 1 states that the Course, Instructor and Enrollment objects were created “in the same way” as Student. Their definition pages are not included as screenshots, but the objects are clearly present in the later evidence: Enrollment carries the three lookup fields, Course and Instructor are chosen as related objects, and all four objects are used in validation rules and flows. Their settings should therefore match the Student object (record name of type Text, reports, activities and search enabled, status Deployed).

Object	Record name (standard Name field)	Notes
Course__c	Course Name (Text)	**To confirm** Confirm plural label “Courses” and options
Instructor__c	Instructor Name (Text)	**To confirm** Confirm plural label “Instructors” and options
Enrollment__c	Enrollment Name (Text; spelling to confirm)	**To confirm** See spelling note above

5. Creating Fields

Every object receives a standard **Name** field when it is created. Additional **custom fields** (API names ending in **__c**) hold the remaining data. The tables below list each field with its label, API name, data type, required status, purpose and any validation rule. Rule IDs refer to Section 7 (VR-1 and VR-2 are configured; VR-P1 to VR-P4 are proposed).

How to read the tables. Field labels and data types come from the Phase 1 field tables. An API name is marked *Confirmed* only when it appears in a screenshot or in the Phase 1 formulas and flow steps; otherwise it is a naming-convention proposal marked *API to confirm*. Where the evidence does not say whether a field is required, the entry reads *To confirm* rather than a guess.

5.1 Student Object (Student__c)

Field label	API name	Data type	Required?	Purpose	Rule	Status
Student Name	Name (standard)	Text (standard)	Required	Record name that identifies each student	—	**Confirmed**
Email	Email__c	Email	To confirm	Institutional e-mail; used to notify the student	VR-1	**Confirmed**
Phone	Phone__c	Phone	To confirm	Student contact number	VR-P2	**API to confirm**
Date of Birth	Date_of_Birth__c	Date	To confirm	Student's date of birth	VR-P1	**API to confirm**
Student Status	Status__c	Picklist	To confirm	Tracks the student's state; the value *Active* is used by the flows and the status report	—	**Confirmed** **To confirm**
Address	Address__c	Text Area	To confirm	Postal address of the student	—	**API to confirm**
Last Name	To confirm	Text	To confirm	A "Last Name" column appears in a Phase 1 report but is not in the Phase 1 field table	—	**To confirm**

The Phase 1 field table calls the picklist *Student Status*, while the flow and report screenshots show *Status* (API name Status__c). The API name is consistent; the label should be aligned.

5.2 Course Object (Course__c)

Field label	API name	Data type	Required?	Purpose	Rule	Status
Course Name	Name (standard)	Text (standard)	Required	Record name that identifies each course	—	**Confirmed**
Category	Category__c	Picklist	To confirm	Course group used to decide the instructor: Technical, Language or Non-Technical	—	**Confirmed**
Duration	Duration__c	Number / Text (Phase 1 lists both)	To confirm	Length of the course; the exact data type and unit must be decided	—	**API to confirm** **To confirm**
Course Fee	Course_Fee__c	Currency	To confirm	Fee charged for the course; revenue reporting refers to this field	VR-P3	**API to confirm**
Description	Description__c	Text Area	Optional (assumed) – confirm	Short description of the course content	—	**API to confirm**

5.3 Instructor Object (Instructor__c)

Field label	API name	Data type	Required?	Purpose	Rule	Status
Instructor Name	Name (standard)	Text (standard)	Required	Record name that identifies each instructor	—	Confirmed
Instructor Code	Instructor_Code__c	Text	To confirm	Short code (for example INSTR_A, INSTR_B, INSTR_C) that lets the flow find the right instructor	—	API to confirm
Expertise	Expertise__c	Picklist	To confirm	Subject area of the instructor	—	API to confirm
Phone	Phone__c	Phone	To confirm	Contact number	—	API to confirm
Email	Email__c	Email	To confirm	Contact e-mail address	—	API to confirm

5.4 Enrollment Object (Enrollment__c)

Field label	API name	Data type	Required?	Purpose	Rule	Status
Enrollment Name	Name (standard)	Text (standard)	Required	Record name of each enrollment (spelled “Enrolement Name” in screenshots)	—	Confirmed To confirm
Student	Student__c	Lookup (Student)	Required	Links the enrollment to one student	—	Confirmed
Course	Course__c	Lookup (Course)	Required	Links the enrollment to one course	—	Confirmed
Instructor	Instructor__c	Lookup (Instructor)	Required	Instructor assigned to this enrollment	—	Confirmed
Enrollment Date	Enrollment_Date__c	Date	To confirm	Set automatically to today by the SetEnrollmentDate flow	—	Confirmed
Enrollment Status	Enrollment_Status__c	Picklist	Required (layout shows required marker)	Lifecycle status: Requested, Approved, Rejected	VR-2	Confirmed
Fees Paid	Fees_Paid__c	Checkbox	Optional (checkbox, default unchecked)	Shows whether the fee is paid; must be ticked before approval	VR-2	Confirmed
Total Amount	Total_Amount__c	Currency	To confirm	Total payable for the enrollment; not visible in the layout screenshots	—	API to confirm To confirm
Comments	Comments__c	Text Area	To confirm	Free-text notes about the enrollment	—	API to confirm
Course Fee	Course_Fee__c (to confirm)	Read-only (lock icon in layout; likely a formula)	Read-only	Displays the fee of the selected course; used by the revenue report	—	To confirm

Picklist values found in the evidence

Picklist	Values confirmed by the evidence	Still to confirm
Enrollment_Status__c	Requested (approval entry criteria), Approved and Rejected (field updates)	Any additional values, default value, restricted list setting
Category__c (Course)	Technical, Language, Non-Technical (instructor assignment rules)	Any additional categories
Status__c (Student)	Active (used by the flow and the status report)	Other values such as Inactive
Expertise__c (Instructor)	—	All values

Design point to resolve. The lookup Instructor__c is set to “Always require a value”, but the instructor is meant to be filled in automatically by a flow that runs after the record is saved. As configured, a user must pick an instructor before saving, which makes the auto-assignment redundant. The team should decide whether Instructor__c stays required (manual choice, flow only corrects it) or becomes optional (flow fills it).

6. Configuring Relationships

6.1 Lookup Relationships and the Junction Object

A **lookup relationship** is a link from a child record to a parent record. The child object holds a lookup field that stores the parent's record ID, and the parent automatically gains a related list of its children. A **junction object** is an object that sits between two other objects to model a many-to-many relationship.

In SECMS, a student can enrol in many courses and a course can have many students. The **Enrollment** object resolves this many-to-many relationship: each enrollment record points to exactly one Student and one Course. Enrollment also points to one Instructor, so instructor workload can be reported through the same records.

The approved Phase 1 design uses **lookup** relationships for all three links. (A master-detail relationship is the other option for a formal junction object; it was not chosen, so roll-up summary fields are not available and child records are not deleted with their parents.)

6.2 Relationship Summary

Child object	Lookup field (API)	Parent object	Required	If parent is deleted	Child relationship name	Status
Enrollment__c	Student__c	Student__c	Yes ("Always require a value" ticked)	Deletion not allowed while enrollments exist	Enrollments	**Confirmed**
Enrollment__c	Course__c	Course__c	Yes	Clear-value option disabled because the field is required; deletion behaviour to confirm	Enrollments	**Confirmed** **To confirm**
Enrollment__c	Instructor__c	Instructor__c	Yes	Deletion not allowed while enrollments exist	Enrollments	**Confirmed**

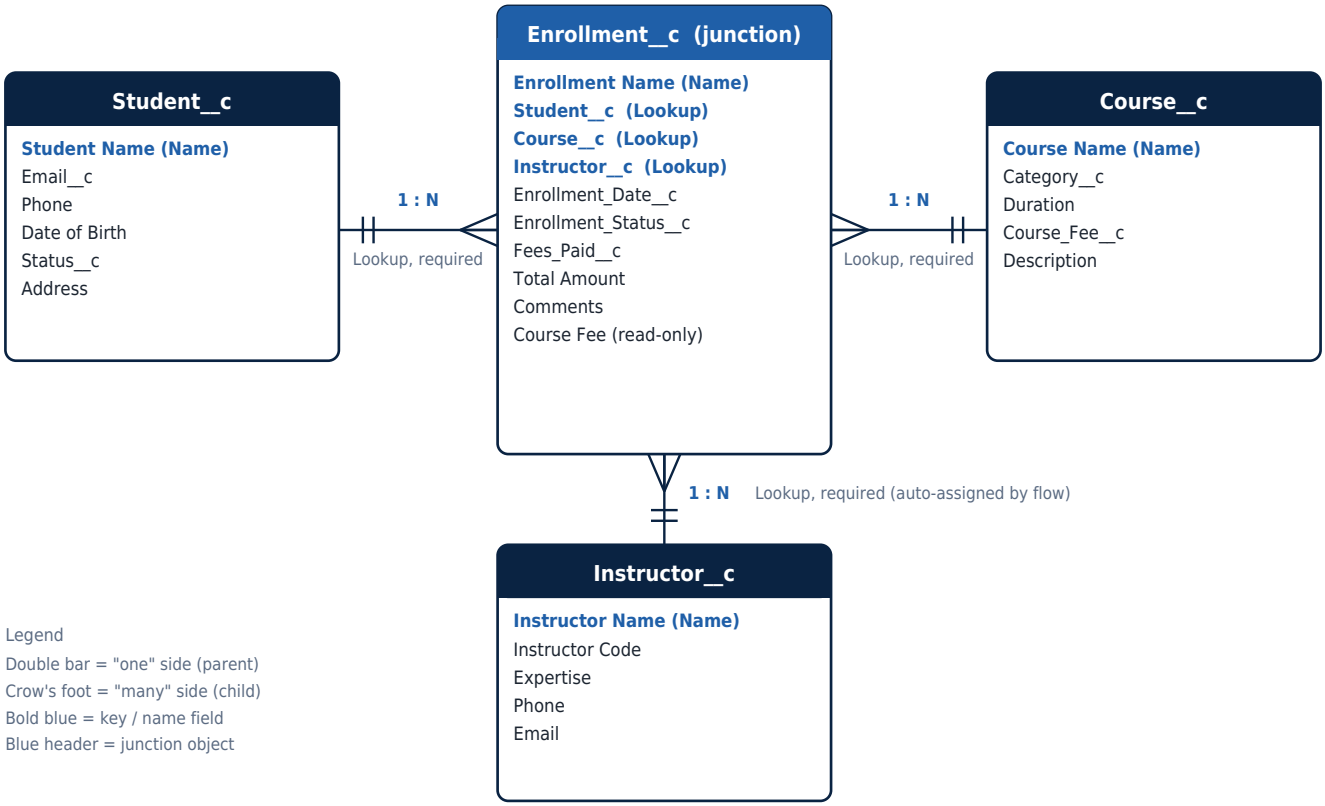


Figure 6: Relationship diagram: Enrollment is the junction between Student and Course and also references Instructor

6.3 Creating the Lookup Relationships

The same procedure is repeated for each parent object (Phase 1, Milestone 4). Each screenshot below shows one of the lookups.

Step	Student lookup	Course lookup	Instructor lookup
1	Setup > Object Manager > Enrollment > Fields & Relationships > New	Same path	Same path
2	Data type: Lookup Relationship, Next	Same	Same
3	Related To: Student	Related To: Course	Related To: Instructor
4	Field Label: Student (API Student__c)	Field Label: Course (API Course__c)	Field Label: Instructor (API Instructor__c)
5	Child Relationship Name: Enrollments; tick “Always require a value”; keep “Don’t allow deletion of the lookup record”	Same settings	Same settings; add to Instructor layout related list
6	Next through Field-Level Security and layout options, then **Save**	Same	Same

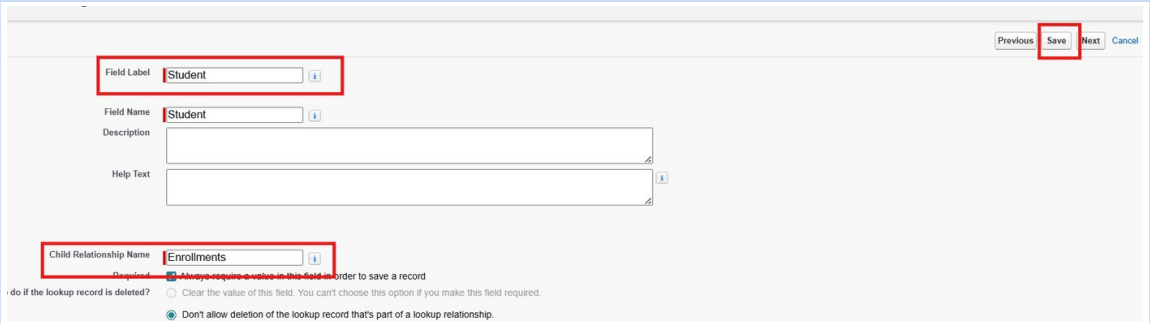


Figure 7: Student lookup: label, child relationship name, required and delete options (Source: project screenshot)



Figure 8: Course lookup: field label and child relationship name (Source: project screenshot)

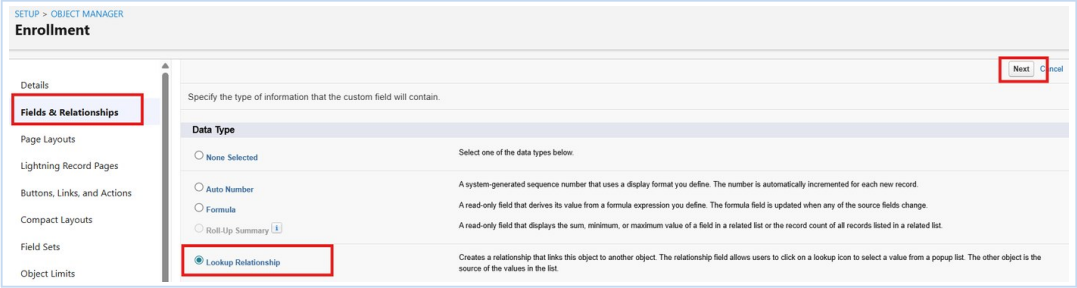


Figure 9: Instructor lookup: choosing the Lookup Relationship data type (Source: project screenshot)

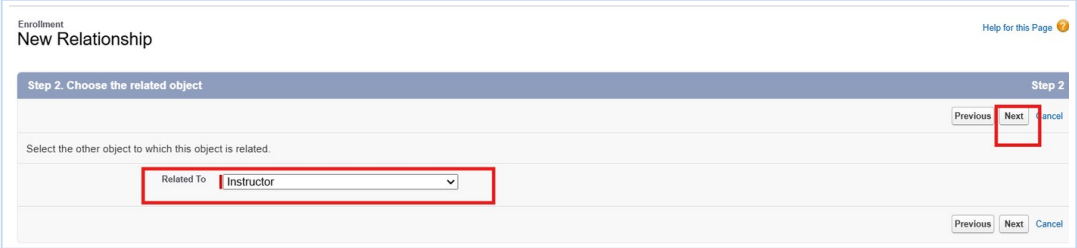


Figure 10: Instructor lookup: selecting the related object (Source: project screenshot)



Figure 11: Instructor lookup: label, child relationship name and required setting (Source: project screenshot)

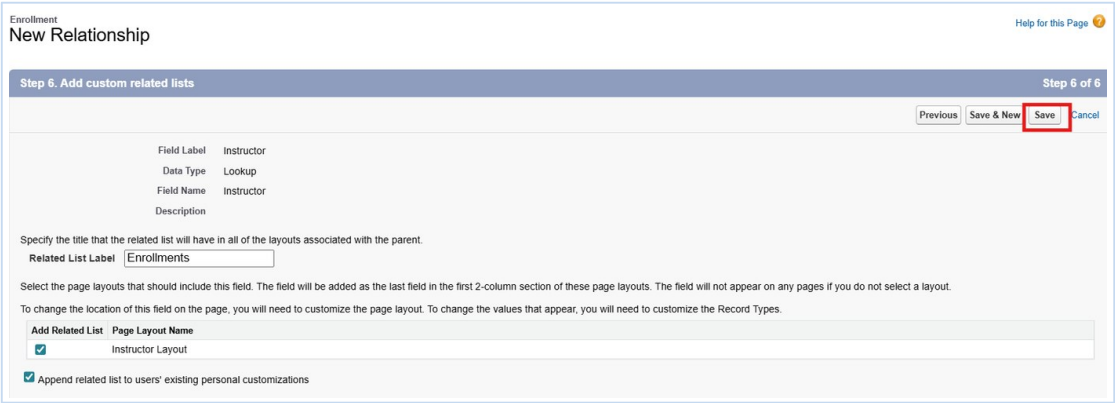


Figure 12: Instructor lookup: final step before saving (Source: project screenshot)

6.4 How the Relationships Support Data Integrity

- **No orphan enrollments:** because Student, Course and Instructor are required, an enrollment cannot be saved without all three parents.
- **No accidental deletion:** the option “Don’t allow deletion of the lookup record that’s part of a lookup relationship” prevents deleting a student or instructor who still has enrollments.
- **Single source of truth:** student, course and instructor details are stored once and referenced by ID, so a change (for example a new phone number) appears everywhere.
- **Reliable reporting:** reports such as Enrollments by Course can group by *Course__r.Name* because every enrollment is linked to a course.

7. Validation Rules

7.1 How Validation Rules Maintain Accurate Data

A **validation rule** contains a formula that returns TRUE or FALSE. Each time a record is saved, Salesforce evaluates the formula; if it returns **TRUE**, the save is blocked and the configured error message appears, either at the top of the page or under a specific field. In other words, the formula describes the *bad* data condition. Validation rules protect data at the moment of entry and apply equally to records created from the UI, imports and integrations.

7.2 Rules Configured in the Salesforce Org

ID	Object	Rule name	Error condition formula	Error message and location	Status
VR-1	Student__c	Screenshot: Email_Must_Be_Student_Domain Phase 1 text: Student_Email_Domain_Validation	`NOT(CONTAINS(Email__c, "@student.edu"))`	Email must be a valid student institution email. Shown on the Email field	**Confirmed** **To confirm**
VR-2	Enrollment__c	Fees_Must_Be_Paid_Before_Approval	`AND(ISPICKVAL(Enrollment_Status__c, "Approved"), Fees_Paid__c = FALSE)`	Fees must be paid before approving an enrollment. Shown at the top of the page	**Confirmed**

To confirm: the VR-1 screenshot shows the rule name *Email_Must_Be_Student_Domain*, but the Phase 1 steps name it *Student_Email_Domain_Validation*. The screenshots show the rule definition pages with the Active box ticked; a Validation Rules list view showing both rules saved and active should be added as verification evidence. The domain *@student.edu* is the example used in Phase 1; the institution's real domain must be confirmed.

VR-1 Student e-mail domain

Purpose: ensure that students use only the allowed institutional e-mail address. **Behaviour:** if the e-mail does not contain @student.edu, saving fails. An empty e-mail also fails (CONTAINS returns FALSE on a blank value), so the rule effectively makes the e-mail mandatory.

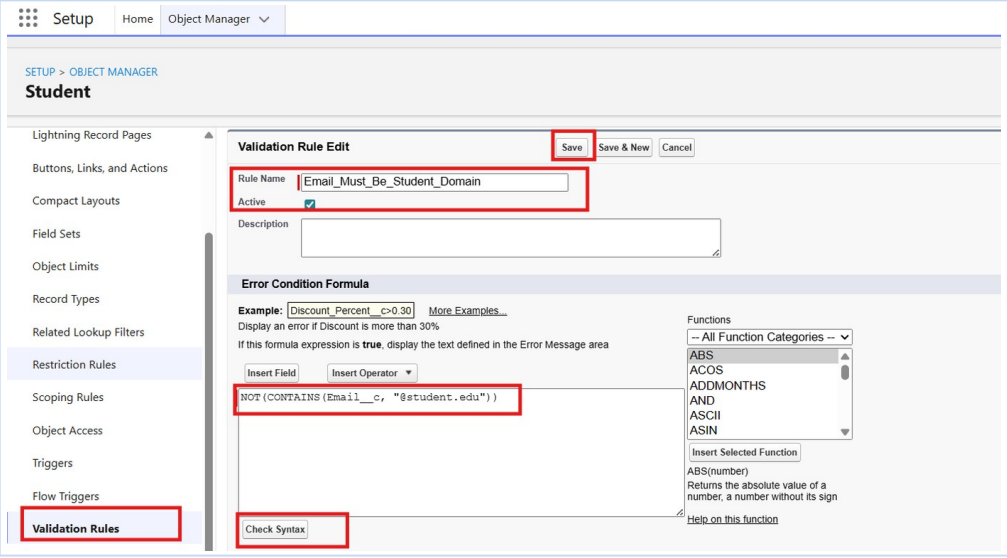


Figure 13: VR-1 rule definition on the Student object (Source: project screenshot)

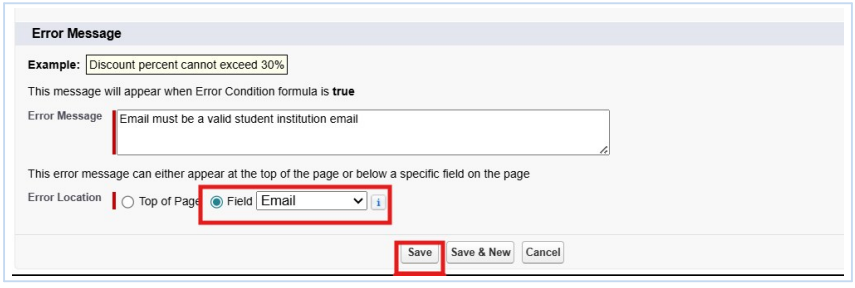


Figure 14: VR-1 error message and error location (Field: Email) (Source: project screenshot)

VR-2 Fees must be paid before approval

Purpose: prevent an enrollment from being approved until the fee is paid. **Behaviour:** the error fires only when the status is Approved *and* Fees Paid is unticked; Requested and Rejected enrollments can still be saved without payment.

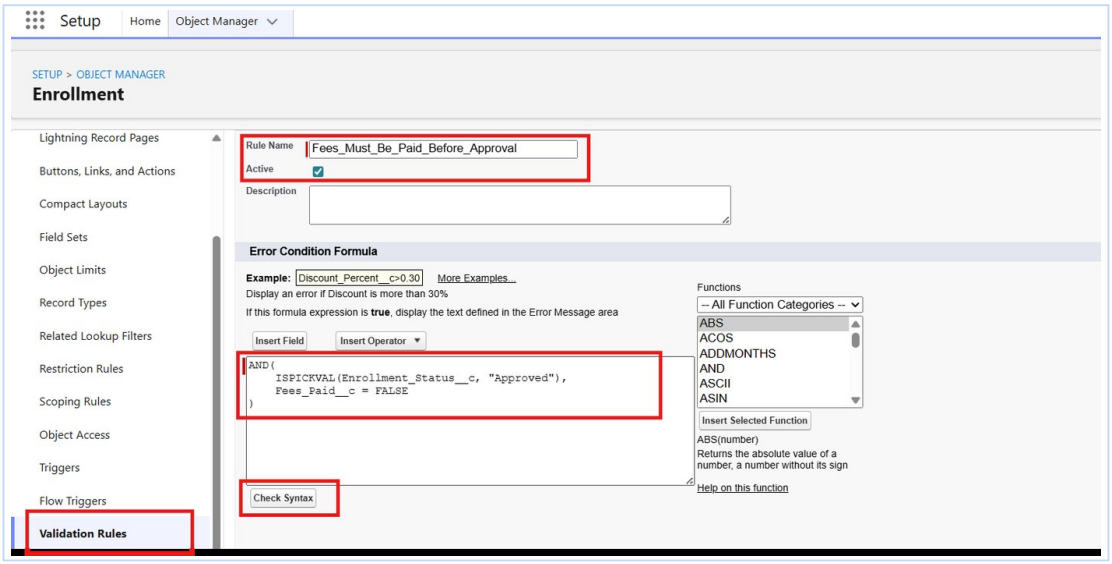


Figure 15: VR-2 rule definition on the Enrollment object (Source: project screenshot)

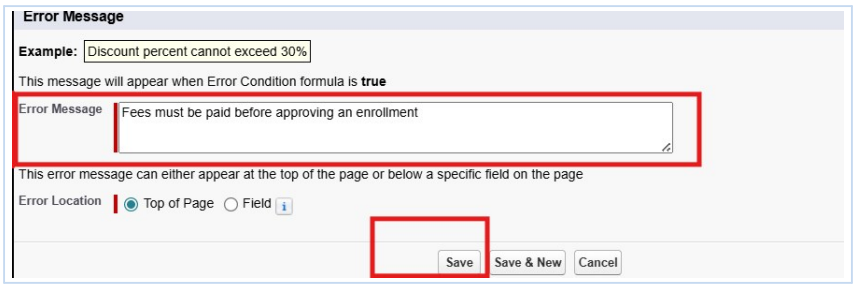


Figure 16: VR-2 error message and error location (Top of Page) (Source: project screenshot)

7.3 Proposed Additional Rules (Not Yet Configured)

The following rules are suggestions for enrollment data, required information and valid field values. They are **not** evidenced as built and must be approved by the team, and their syntax checked with *Check Syntax* in Setup, before use.

ID	Object	Goal	Proposed error condition formula	Proposed error message	Status
VR-P1	Student__c	Valid date of birth	`Date_of_Birth__c > TODAY()`	Date of birth cannot be in the future.	**Proposed**
VR-P2	Student__c	Valid phone format	`AND(NOT(ISBLANK(Phone__c)), NOT(REGEX(Phone__c, "^\+?[0-9]{10,13}$")))`	Enter 10 to 13 digits, with an optional leading +.	**Proposed**
VR-P3	Course__c	Valid fee	`Course_Fee__c < 0`	Course fee cannot be negative.	**Proposed**
VR-P4	Enrollment__c	Valid enrollment date	`Enrollment_Date__c > TODAY()`	Enrollment date cannot be in the future.	**Proposed**

7.4 Suggested Refinement of VR-1

CONTAINS accepts the domain anywhere in the text (for example *a@student.edu.example.com*) and is case-sensitive, so *@Student.edu* would be rejected. A stricter proposed version checks the end of the value in lower case:

```
NOT(RIGHT(LOWER(Email__c), 12) = "@student.edu")
```

This is a proposal only; the configured rule remains as shown in Section 7.2.

7.5 Duplicate Enrollment Control (Proposed)

Phase 1 (NFR-3) requires “no duplicate course registration”. A validation rule cannot compare a record with other records, and no duplicate-prevention control appears in the evidence. One proposed approach: add a text field *Unique_Key__c* marked **Unique** (case-insensitive) on Enrollment, filled by a flow with the Student ID and Course ID joined together. A second enrollment for the same pair would then be rejected. This item is **proposed** and has not been built.

8. Record-Triggered Flows

8.1 Backend Automation Overview

A **record-triggered flow** runs automatically when a record is created, updated or deleted. It starts with an *object*, a *trigger* and optional *entry conditions*, and then runs elements such as **Get Records** (read data), **Decision** (choose a path), **Update Records**, **Create Records** and **Action** (for example an e-mail alert). The SECMS flows all run on the Enrollment object and are optimised for *Actions and Related Records*, which means they run after the record is saved.

Flow (saved name)	Object / trigger	Entry condition	Purpose	Evidence and status
SetEnrollmentDate	Enrollment; record is created	None	Set Enrollment_Date__c to today	Element screenshots. Activation not shown
Enrollment Approved Action	Enrollment; created or updated (screenshot). Phase 1 text says “on update”	Enrollment Status equals Approved	Send approval e-mail, notify Admin if needed, update the student status	Element screenshots. Activation not shown. Several items to confirm (8.3)
AutoAssign_Instructor (Flow Builder title: AutoAssign Instructor)	Enrollment; created or updated	1 condition (not readable in the screenshot)	Assign Instructor A, B or C from the Course Category	Flow Builder: **Active**, version V18, last saved 12/26/2025 03:30 PM
Create_Followup_Task	Enrollment; created or updated	Enrollment Status equals Requested	Create a follow-up Task for the Admin due in 2 days	Element screenshots. Activation not shown

Implementation claims: the report describes a flow as *active* only where a screenshot shows the Active badge (AutoAssign_Instructor). For the other flows the screenshots show the configuration but not the activation state, so activation must be confirmed in Setup > Flows.

8.2 Enrollment Record Processing

Flow 1: SetEnrollmentDate

Purpose: stamp every new enrollment with the creation date. **Elements:** (1) Start: object Enrollment, trigger *A record is created*, no entry conditions, optimised for Actions and Related Records. (2) *Update Records* element labelled **Set Enrollment Date** (API Set_Enrollment_Date) that updates the record that triggered the flow. **Assignment logic:** Enrollment_Date__c = *$Flow.CurrentDate* (shown as Running Flow Interview > CurrentDate). Phase 1 saves it as SetEnrollmentDate and activates it.

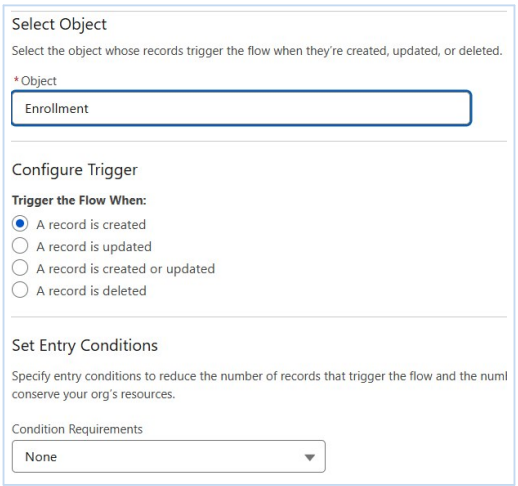


Figure 17: SetEnrollmentDate: object, trigger and entry conditions (Source: project screenshot)

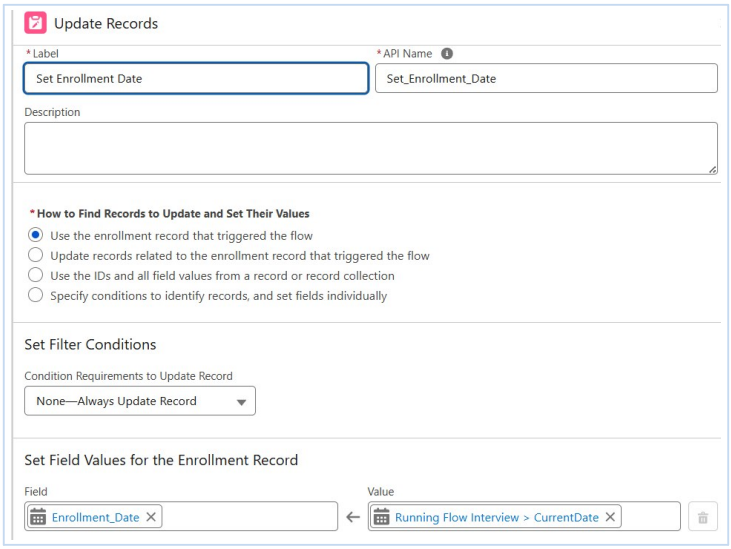


Figure 18: SetEnrollmentDate: Update Records element setting Enrollment_Date (Source: project screenshot)

8.3 Enrollment Status Updates and Approval-Related Automation

Flow 2: Enrollment Approved Action

Purpose: when an enrollment becomes Approved, notify the student, alert the Admin and set the student to Active.
Elements: Start (Enrollment, entry condition *Enrollment Status Equals Approved*, all conditions met) > **Get Records** (Get Student) > **Action** (Send Student Email, using the e-mail alert "Enrollment Approved EmailAlert" with the triggering record ID) > optional Admin notification > **Update Records** (Set Student Status Active).

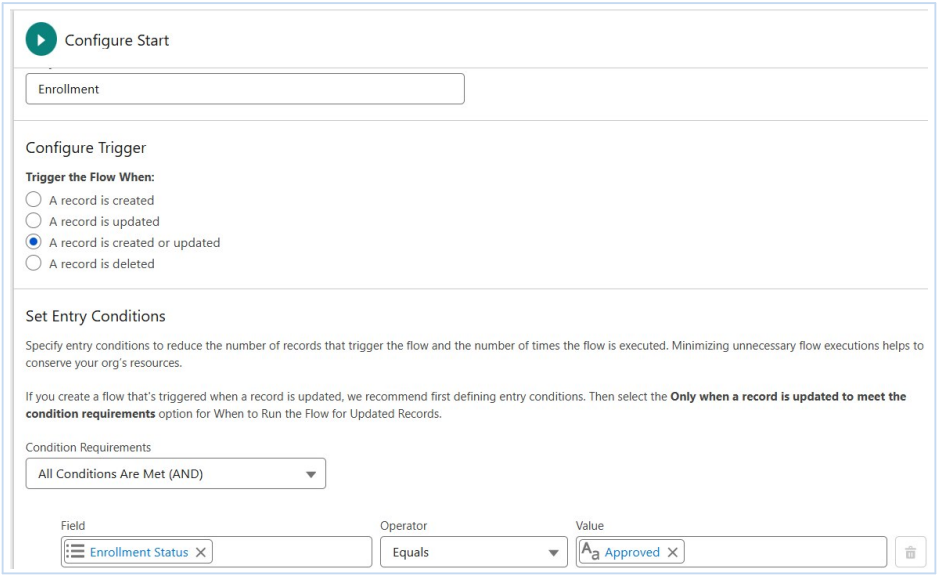


Figure 19: Enrollment Approved Action: start configuration and entry condition (Source: project screenshot)

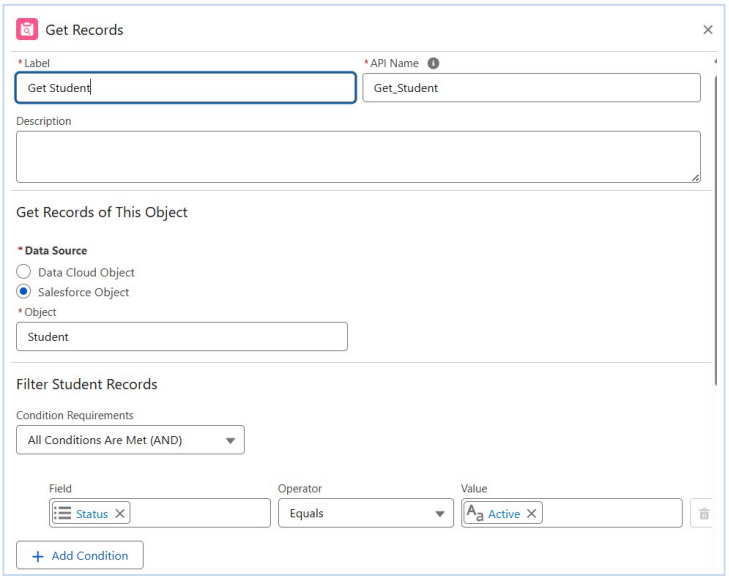


Figure 20: Enrollment Approved Action: Get Records for the Student object (Source: project screenshot)

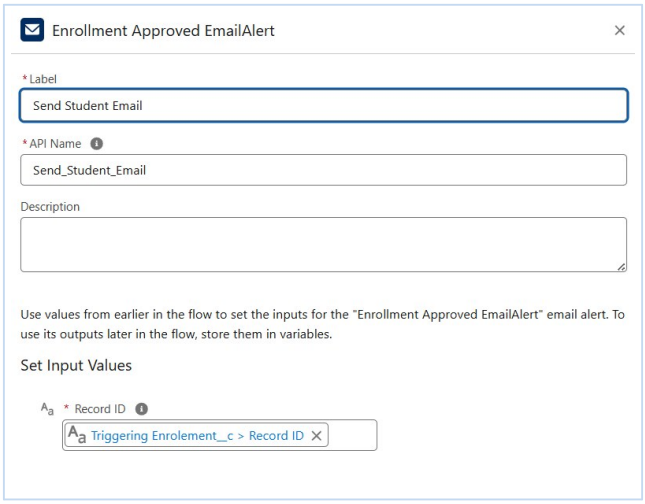


Figure 21: Enrollment Approved Action: e-mail alert action (Source: project screenshot)

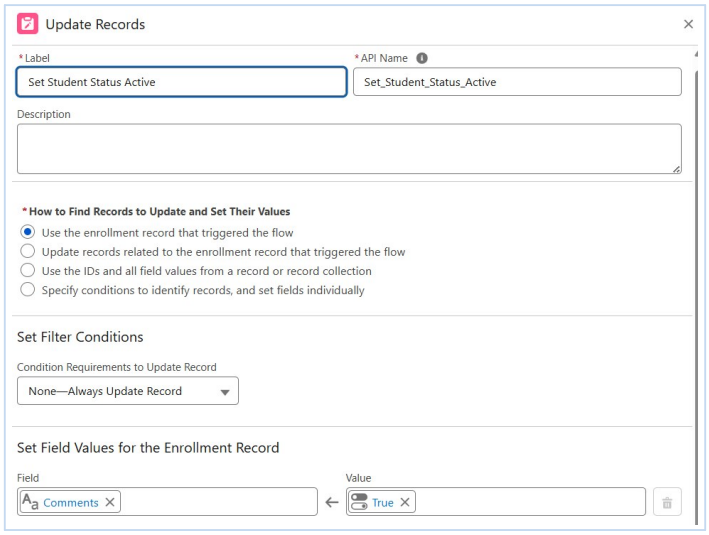


Figure 22: Enrollment Approved Action: Update Records element (Source: project screenshot)

Points the team must review (To confirm):

- The Get Student element in the screenshot filters Student records with *Status equals Active* only. It does not use the student linked to the enrollment (for example Student__c of the triggering record), so it may return the wrong student.

- The Update Records element is named “Set Student Status Active”, but the screenshot shows it updating the *enrollment record that triggered the flow* and setting the field *Comments* to *True*. The Phase 1 text says the flow should set *Student__c.Status__c = Active*. The element must be corrected so that it updates the student.
- The trigger is *created or updated* in the screenshot but “trigger on update” in the Phase 1 steps. Choose one, and enable “Only when a record is updated to meet the condition requirements” so the e-mail is not sent on every later save.

Approval process: Approval_Enrollment_Request

Purpose: route enrollment requests to the Training Manager for approval or rejection. The screenshots show the following configuration (Phase 1, Milestone 6):

Component	Configuration	Status
Process	Approval Enrollment Request (unique name Approval_Enrollment_Request) on the Enrollment object, created with the Standard Setup Wizard; description: sends to Training Manager	**Confirmed**
Entry criteria	Enrollment: Enrollment Status equals Requested	**Confirmed**
Approval step	Step named Training Manager; approver is a specific Training Manager user	**Confirmed**
Final Approval Action	Field Update **Status to Approved** (unique name Status_to_Approved) sets Enrollment Status to Approved	**Confirmed**
Final Rejection Actions	Field Update **Set Status to Rejected** sets Enrollment Status to Rejected; Email Alert **Enrollment Rejected** (template “Enrollment Rejected”) notifies the student	**Confirmed**
Activation	Phase 1 step 13 says to activate the process; an activation screenshot is not included	**To confirm**

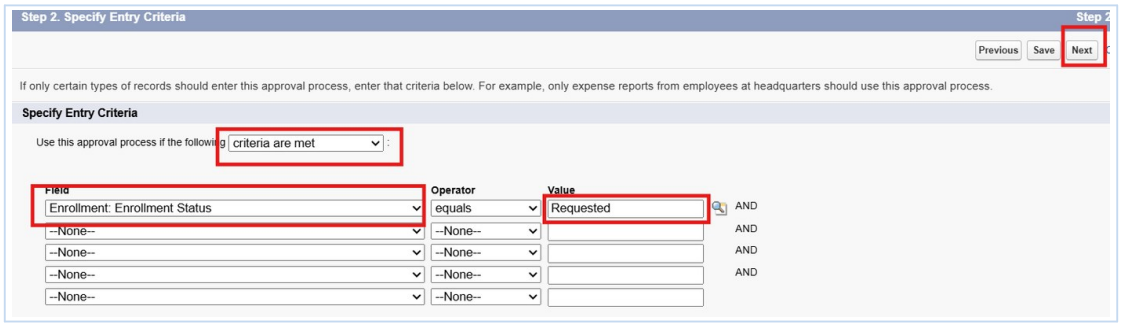


Figure 23: Approval process: entry criteria (Enrollment Status equals Requested) (Source: project screenshot)

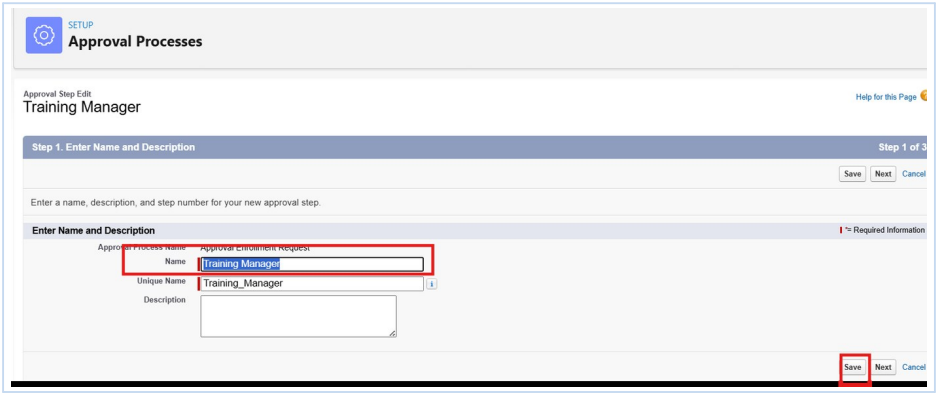


Figure 24: Approval process: Training Manager approval step (Source: project screenshot)

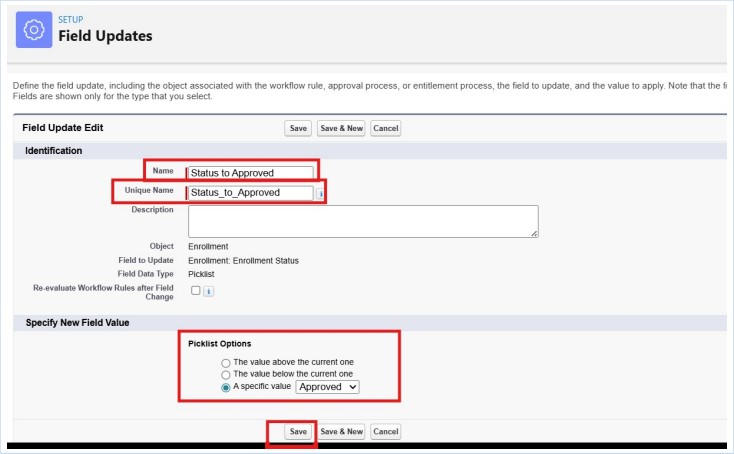


Figure 25: Final approval action: field update to Approved (Source: project screenshot)

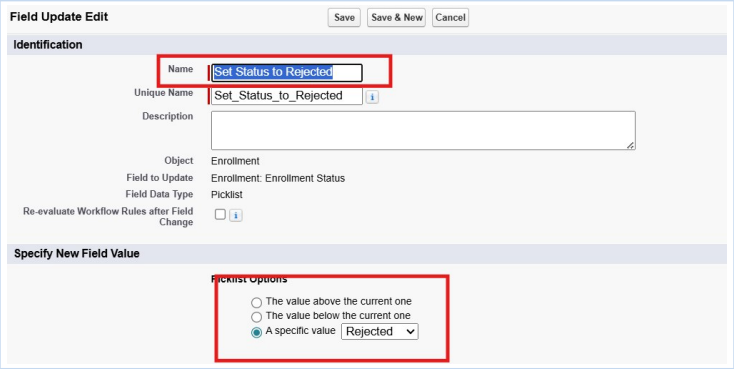


Figure 26: Final rejection action: field update to Rejected (Source: project screenshot)

Interaction to test: VR-2 blocks the value Approved when Fees Paid is unticked. The approval process's final action also sets the status to Approved. The team should test that an approver cannot complete an approval for an unpaid enrollment (or decide where the fee check should happen) – see test cases TC-24 to TC-26.

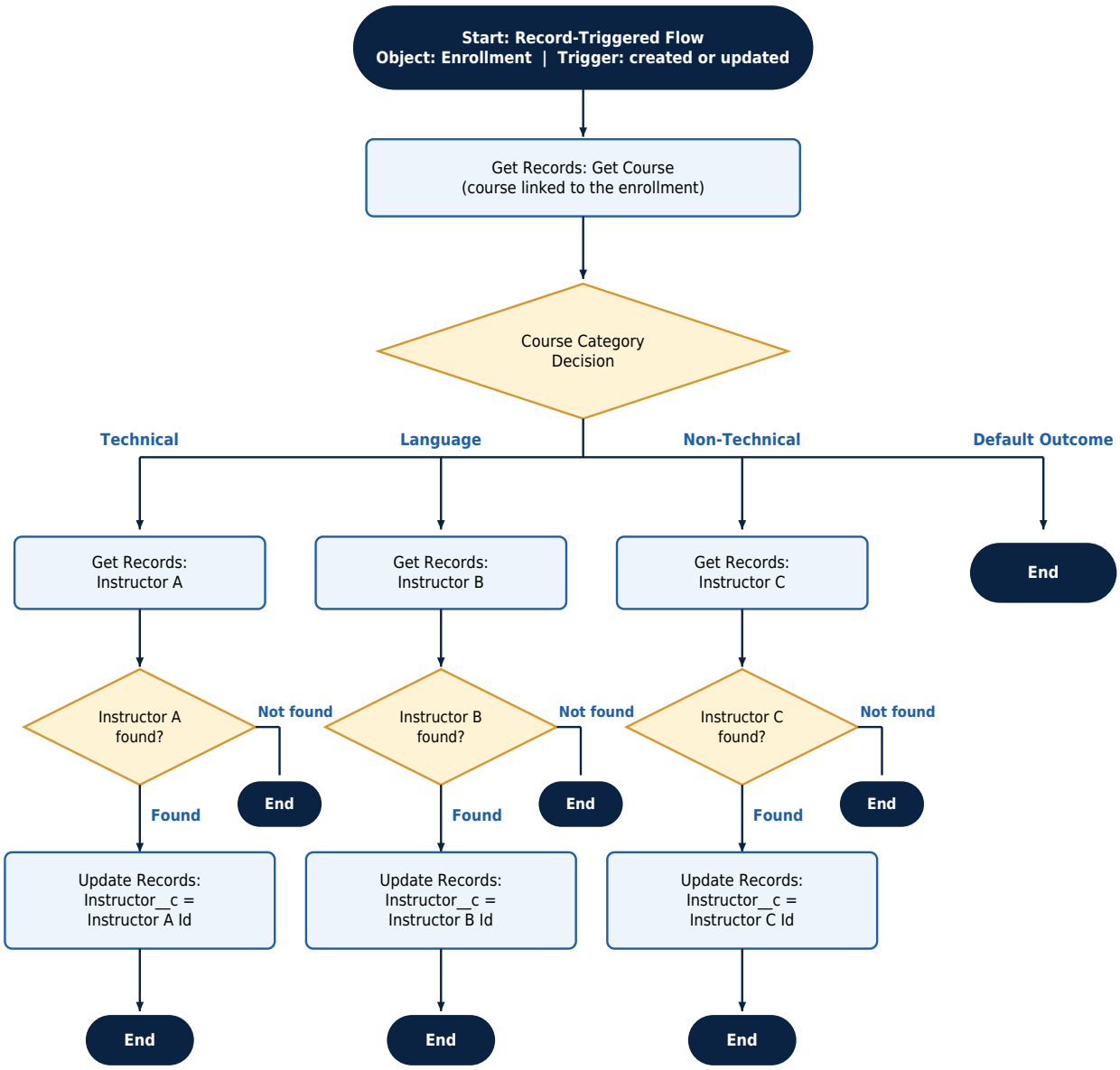
8.4 Instructor Assignment Based on Course Category

Purpose: assign an instructor automatically according to the category of the enrolled course. The approved rules are:

Course category (Category__c)	Instructor assigned	Instructor record lookup (Phase 1)
Technical	Instructor A	Instructor__c record with code INSTR_A (or by name)
Language	Instructor B	Instructor__c record with code INSTR_B (or by name)
Non-Technical	Instructor C	Instructor__c record with code INSTR_C (or by name)

Flow elements and logic (AutoAssign_Instructor):

- **Start:** Enrollment, trigger *A record is created or updated*, optimised for Actions and Related Records; the Start box shows 1 entry condition.
- **Get Course** (Get Records): reads the Course linked to the enrollment.
- **Course Category Decision:** outcomes *Technical*, *Language*, *NonTechnical* and a *Default Outcome*.
- **Get Instructor A / B / C** (Get Records): one per outcome, finding the instructor by code or name.
- **Instructor found? decision:** "Found" continues; "Not Found" (or default) goes straight to End so that no blank value is written.
- **Assign Instructor A / B / C** (Update Records): sets *Instructor__c* on the triggering enrollment to the Id of the instructor found. Every path finishes at an End element.



Redrawn from the Flow Builder screenshot (AutoAssign Instructor, V18). Rules: Technical > Instructor A, Language > Instructor B, Non-Technical > Instructor C. A category that matches none of the three follows the Default Outcome and ends without an update.

Figure 27: Flowchart of the AutoAssign_Instructor record-triggered flow

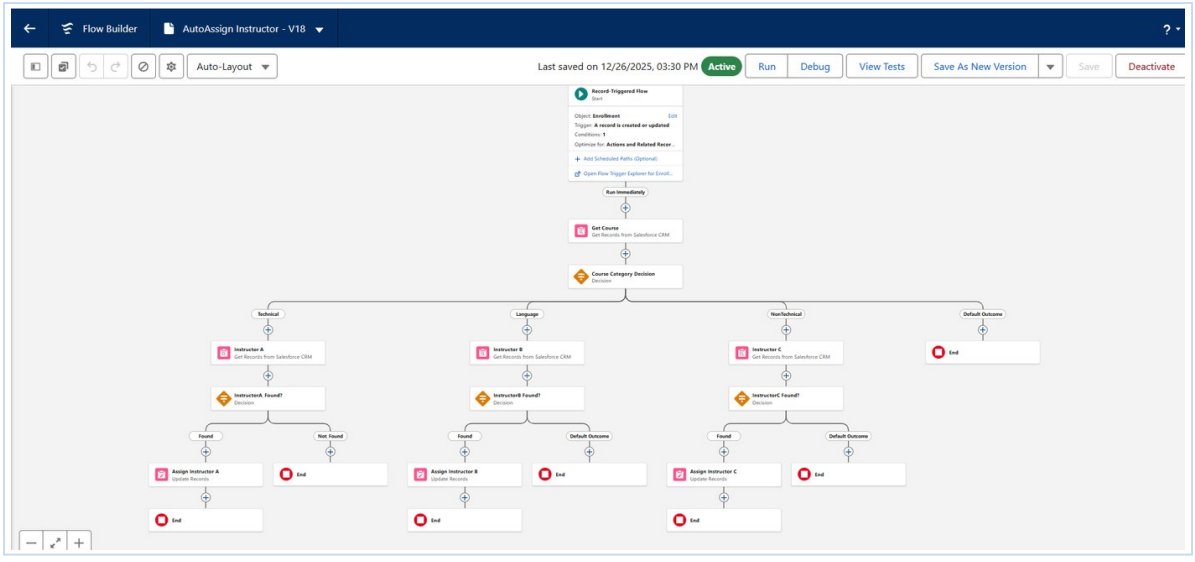


Figure 28: Flow Builder canvas: AutoAssign Instructor, version V18, status Active (Source: project screenshot)

Recommendation (Proposed): because the flow runs on every create *and* update, it will overwrite a manually chosen instructor whenever the enrollment is edited. Adding an entry condition such as “Instructor is null”, or a check that the Course changed, would avoid this. The interaction with the required Instructor__c field is described in Section 5.4.

8.5 Follow-Up Task Creation

Flow 4: Create_Followup_Task

Purpose: when an enrollment is Requested, create a follow-up task for the Admin. **Elements:** Start (Enrollment, created or updated, condition *Enrollment Status equals Requested*) > **Get Records** (Get Admin User from the User object, filtered by Username; the username is not shown here) > **Formula** (Date) *DueDateTodayPlus2* = TODAY() + 2 > **Create Records** (Task).

Task field	Value assigned in the flow
Subject	Follow up on enrollment request
Description	Auto-created follow-up for enrollment request
Due Date Only	DueDateTodayPlus2 (today + 2 days)
Priority / Status	Normal / Not Started
Related To ID (WhatId)	Record ID of the triggering Enrollment
Owner	Admin user found by Get Admin User (Phase 1 step 7)

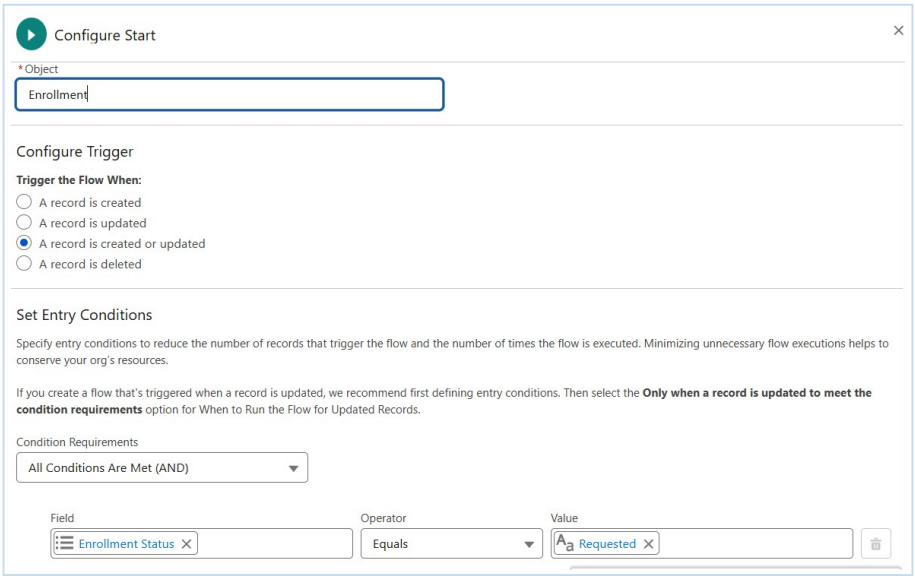


Figure 29: Create_Followup_Task: start configuration (Source: project screenshot)

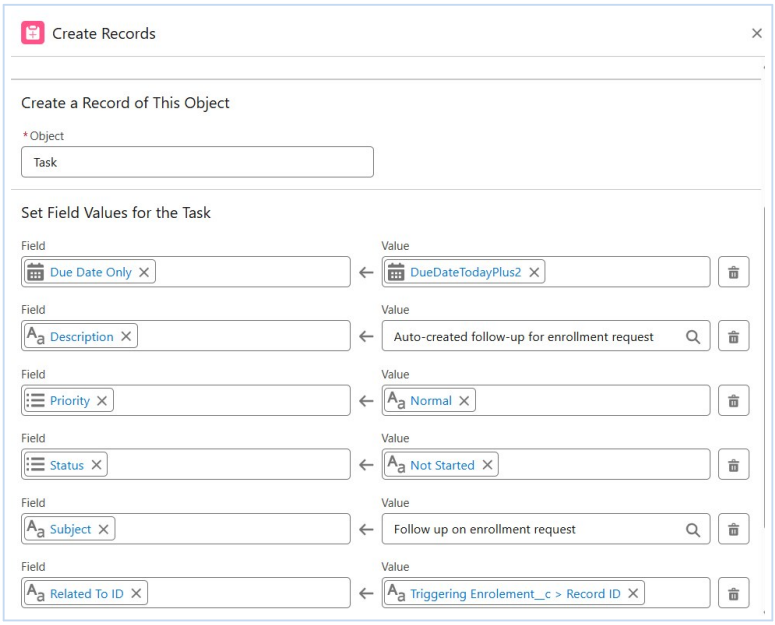


Figure 30: Create_Followup_Task: Create Records element for the Task (Source: project screenshot)

Review point (To confirm): the flow runs on create *or* update. Because AutoAssign_Instructor updates the same record, the follow-up flow may run more than once for a Requested enrollment and create duplicate tasks. Selecting “Only when a record is updated to meet the condition requirements” is proposed.

9. Backend Business Logic

The objects, fields, relationships, validation rules, flows and the approval process form one chain. Each layer does a different job, and the layers depend on each other:

Layer	Role in the enrollment process	Data integrity benefit
Objects	Student, Course, Instructor and Enrollment separate the different kinds of academic data	Each fact is stored in one place
Fields	Capture details (e-mail, category, fee) and control values (status, fees paid)	Picklists limit values to a defined list
Relationships	Enrollment references one Student, one Course and one Instructor	No orphan enrollments; parents cannot be deleted while in use
Validation rules	Block invalid e-mail domains and approvals without fee payment	Bad data is stopped at entry
Record-triggered flows	Set the enrollment date, assign the instructor, create the Admin follow-up task and react to approval	Reduces manual work and human error
Approval process	Sends Requested enrollments to the Training Manager and records the decision as Approved or Rejected	Every registration is verified before confirmation

9.1 End-to-End Enrollment Processing Logic

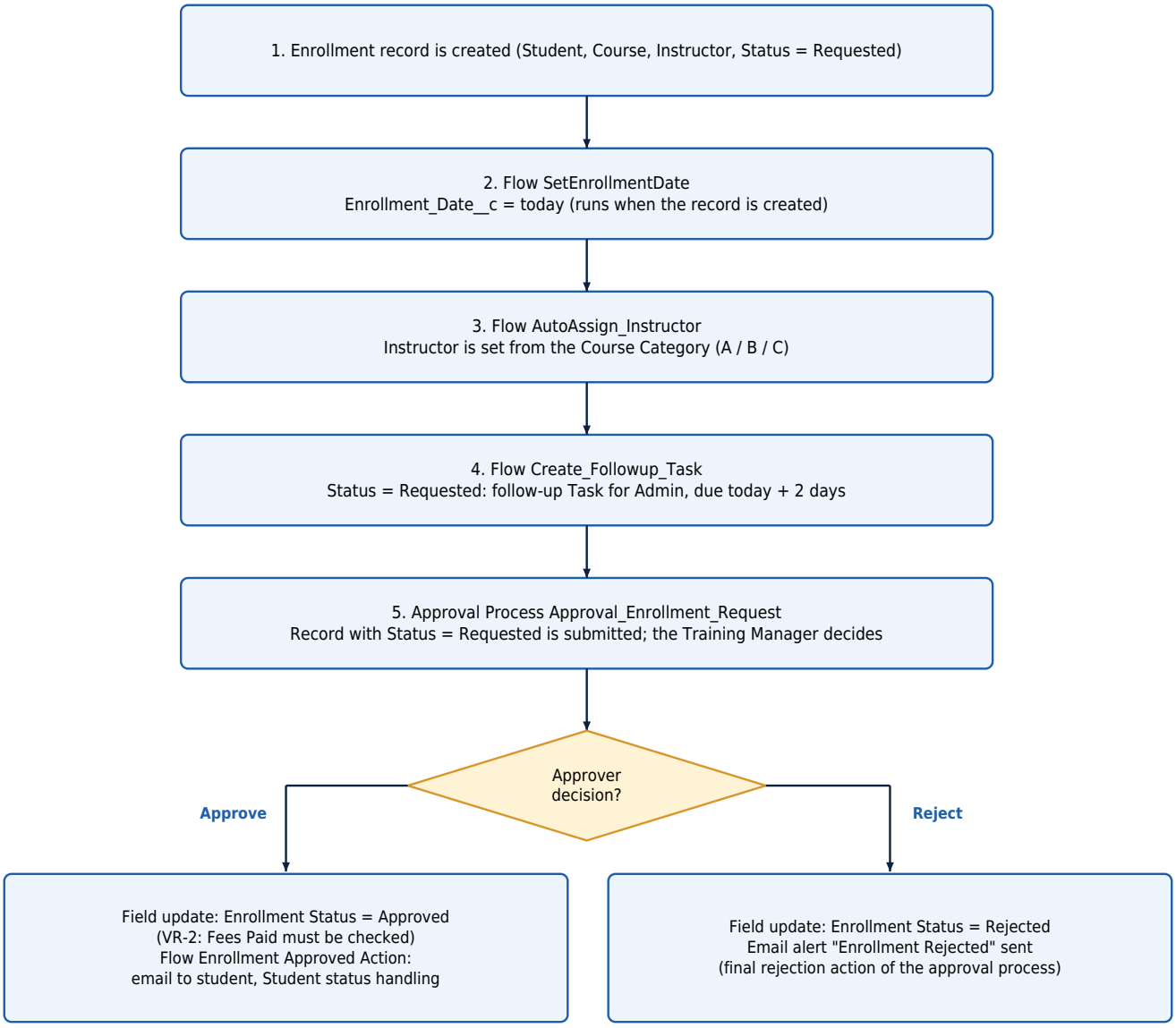


Figure 31: Backend processing sequence for one enrollment (configuration described in Sections 6 to 8)

9.2 How the Components Work Together

- A **Requested** enrollment is the starting point: the approval process and the follow-up task both watch for this status.
- The **lookups** supply the data used by flows: the Course lookup lets AutoAssign_Instructor read the category; the Student lookup lets the approval flow reach the student.
- The **validation rules** guard the two data entry points that matter most: the student's e-mail (used for notifications) and the status change to Approved (linked to fees).
- The **approval process** is the only path that moves a Requested enrollment to Approved or Rejected in the automated design; the e-mail alert then informs the student of a rejection, and the approval flow informs the student of an approval.
- The stored, clean data is what the reports and dashboards of a later phase will read.

10. Testing and Verification

The test cases below verify the Phase 2 backend. **Actual Result** and **Status** are intentionally left as "To be recorded" and "Pending" until the team runs each test in the org and attaches a screenshot as evidence. Cases marked with an asterisk (*) test an item that is proposed or unresolved and may be expected to fail until the design decision is made.

10.1 Creating Records

Test Case ID	Test Scenario	Expected Result	Actual Result	Status
TC-01	Create a Student with a name and an e-mail ending in @student.edu	Record saves; it appears in the Students list and in a query of Student__c	To be recorded	**Pending**
TC-02	Create a Course with Category = Technical and a course fee	Record saves with the selected category and fee	To be recorded	**Pending**
TC-03	Create Instructor records with codes INSTR_A, INSTR_B and INSTR_C	Three instructor records are saved and can be selected in the Instructor lookup	To be recorded	**Pending**
TC-04	Create an Enrollment with a Student, a Course, an Instructor and Status = Requested	Record saves; Enrollment_Date__c is filled with today's date by the SetEnrollmentDate flow	To be recorded	**Pending**

10.2 Required Field Validation

Test Case ID	Test Scenario	Expected Result	Actual Result	Status
TC-05	Save a Student without a Student Name	Save is blocked; the required-field message appears on the Name field	To be recorded	**Pending**
TC-06	Save an Enrollment without a Student	Save is blocked with a required-field message on Student	To be recorded	**Pending**
TC-07	Save an Enrollment without a Course	Save is blocked with a required-field message on Course	To be recorded	**Pending**
TC-08*	Save an Enrollment without an Instructor	As configured (required), save is blocked. Record the result and decide whether Instructor__c should be optional so the flow can assign it	To be recorded	**Pending**
TC-09	Save a Student with an e-mail that does not end in @student.edu (or with a blank e-mail)	VR-1 blocks the save; message "Email must be a valid student institution email" appears under the Email field	To be recorded	**Pending**
TC-10	Save a Student with a valid @student.edu e-mail	Record saves without an error	To be recorded	**Pending**

10.3 Validation Rule Behaviour

Test Case ID	Test Scenario	Expected Result	Actual Result	Status
TC-11	Set Enrollment Status = Approved while Fees Paid is unticked	VR-2 blocks the save; “Fees must be paid before approving an enrollment” appears at the top of the page	To be recorded	**Pending**
TC-12	Tick Fees Paid, then set Enrollment Status = Approved	Record saves without an error	To be recorded	**Pending**
TC-13	Set Enrollment Status = Rejected with Fees Paid unticked	Record saves (VR-2 applies only to Approved)	To be recorded	**Pending**

10.4 Relationship Verification

Test Case ID	Test Scenario	Expected Result	Actual Result	Status
TC-14	Open an Enrollment and check that Student, Course and Instructor show the parent record names as links	Each lookup resolves to the correct parent record	To be recorded	**Pending**
TC-15	Run a query or report of Enrollment with Student__r.Name and Course__r.Name	Each enrollment is linked to the expected parents	To be recorded	**Pending**
TC-16	Delete a Student that has at least one Enrollment	Deletion is blocked by the lookup delete setting	To be recorded	**Pending**
TC-17	Delete a Course that has at least one Enrollment	Deletion is blocked (behaviour to be confirmed for Course)	To be recorded	**Pending**
TC-18	Delete an Instructor that has at least one Enrollment	Deletion is blocked by the lookup delete setting	To be recorded	**Pending**

10.5 Instructor Assignment

Test Case ID	Test Scenario	Expected Result	Actual Result	Status
TC-19	Create an Enrollment for a course with Category = Technical	Instructor__c is set to Instructor A	To be recorded	**Pending**
TC-20	Create an Enrollment for a course with Category = Language	Instructor__c is set to Instructor B	To be recorded	**Pending**
TC-21	Create an Enrollment for a course with Category = Non-Technical	Instructor__c is set to Instructor C	To be recorded	**Pending**
TC-22	Create an Enrollment for a course whose category is blank or not one of the three	Default Outcome: the flow ends and no instructor is written	To be recorded	**Pending**
TC-23*	Edit a saved Enrollment whose instructor was changed manually	Record whether the flow overwrites the manual choice (see the recommendation in 8.4)	To be recorded	**Pending**

10.6 Enrollment Status Changes and Approval

Test Case ID	Test Scenario	Expected Result	Actual Result	Status
TC-24	Set the status to Requested, then submit the Enrollment for approval	The request is routed to the Training Manager; a follow-up Task for the Admin exists with due date = today + 2 days	To be recorded	**Pending**
TC-25	The approver approves an enrollment whose Fees Paid is ticked	Status becomes Approved; the student receives the approval e-mail	To be recorded	**Pending**
TC-26*	The approver approves an enrollment whose Fees Paid is unticked	Record the outcome: VR-2 should not allow an Approved enrollment without payment	To be recorded	**Pending**
TC-27	The approver rejects an enrollment	Status becomes Rejected; the “Enrollment Rejected” e-mail alert is sent	To be recorded	**Pending**
TC-28*	After approval, check the student record status	Student status is Active (depends on correcting the Update Records element in Flow 2)	To be recorded	**Pending**
TC-29*	Save a Requested enrollment several times	Only one follow-up Task is created (depends on the flow entry-condition setting)	To be recorded	**Pending**

10.7 Duplicate or Invalid Data

Test Case ID	Test Scenario	Expected Result	Actual Result	Status
TC-30*	Create a second Enrollment for the same Student and Course	Blocked, as NFR-3 requires. No control is evidenced, so this test is expected to fail until 7.5 is implemented	To be recorded	**Pending**
TC-31*	Enter a Date of Birth in the future (after VR-P1 is added)	Save is blocked with “Date of birth cannot be in the future.”	To be recorded	**Pending**
TC-32*	Enter a negative Course Fee (after VR-P3 is added)	Save is blocked with “Course fee cannot be negative.”	To be recorded	**Pending**
TC-33	Insert an Enrollment through Data Import with a Student ID that does not exist	The row is rejected with an invalid-ID error	To be recorded	**Pending**

Verification evidence: after each test, capture a screenshot of the record, the error message or the report, name it with the Test Case ID (for example TC-11.png) and add it to the final submission. No test results are claimed in this version of the document.

11. Phase 2 Deliverables

11.1 Deliverables and Their Status

Deliverable	Description	Evidence available	Status
Configured Salesforce objects	Student__c, Course__c, Instructor__c and Enrollment__c	Student object screenshots; other objects appear in relationship, validation and flow screenshots	**Confirmed** **To confirm**
Configured fields	Fields listed in Section 5	Partial: several fields and API names visible; others from Phase 1 tables only	**API to confirm**
Established relationships	Three lookups from Enrollment to Student, Course and Instructor	Screenshots of all three lookups	**Confirmed**
Validation rules	VR-1 and VR-2 (VR-P1 to VR-P4 proposed)	Rule definition screenshots	**Confirmed**
Record-triggered flows	SetEnrollmentDate, Enrollment Approved Action, AutoAssign_Instructor, Create_Followup_Task	Element screenshots; Active badge only for AutoAssign_Instructor	**Confirmed** **To confirm**
Approval process	Approval_Enrollment_Request with approval and rejection actions	Configuration screenshots	**Confirmed** **To confirm**
Backend configuration documentation	This document	Delivered as a draft for review	**Confirmed**
Test cases and verification evidence	Section 10 test cases; screenshots of results	Test cases written; no results yet	**Pending**

11.2 Items Requiring Confirmation Before Submission

#	Item to confirm in the Salesforce org	Section
1	Exact spelling and API name of the Enrollment object and its record-name label (Enrollment / Enrolement)	4.1
2	API names, data types and required settings of all fields marked “API to confirm” or “To confirm”, including Duration (Number or Text), Total Amount, Comments, Last Name and the Course Fee shown on Enrollment	5
3	Complete picklist values for Enrollment Status, Student Status, Category and Expertise	5.4
4	Whether Instructor__c should stay required, given the auto-assignment flow	5.4, 8.4
5	Delete behaviour of the Course lookup	6.2
6	Saved name of VR-1 (Email_Must_Be_Student_Domain or Student_Email_Domain_Validation) and the real institutional e-mail domain; both rules shown Active in a list view	7.2
7	Activation status of SetEnrollmentDate, Enrollment Approved Action, Create_Followup_Task and the approval process	8
8	Enrollment Approved Action: filter of Get Student, the target of the Update Records element (Comments vs Student status) and the trigger type	8.3
9	Entry conditions of AutoAssign_Instructor and Create_Followup_Task to avoid overwriting instructors and creating duplicate tasks	8.4, 8.5
10	Approval of the proposed rules VR-P1 to VR-P4 and the duplicate-control approach	7.3, 7.5

12. Phase 2 Conclusion

Phase 2 turned the confirmed Phase 1 requirements into the backend of the Student Enrollment & Course Management System on a Salesforce Developer Org. The project evidence shows the org prepared, four custom objects (Student, Course, Instructor and Enrollment) created, three lookup relationships that make Enrollment the junction between students and courses and the link to an instructor, two validation rules that protect e-mail data and fee payment before approval, and four record-triggered flows plus an approval process that automate enrollment date stamping, instructor assignment by course category, follow-up task creation and the approve or reject decision.

The work is not yet fully verified. Some field API names, flow settings and activation states still need to be confirmed in the org, a few design points need a team decision (required Instructor__c, duplicate enrollment control, flow entry conditions), and the test cases in Section 10 must be run so that results and screenshots can be added.

Preparing for Phase 3: UI/UX Development & Customization

Phase 3 builds the user-facing layer on top of the backend documented here. It depends on a stable and verified data model: user screens can only be trusted when the objects, fields, relationships, validation and automation behind them behave correctly. Before Phase 3 starts, the team should close the confirmation items in Section 11.2, execute the Section 10 tests and record the results. Once that is done, the interface can be designed with confidence that the underlying data will remain accurate and consistent.

References

#	Source	How it was used
1	Student Enrollment & Course Management System (SECMS): confirmed Phase 1 project document, including its Salesforce screenshots (provided by the project team)	Main source for requirements, field lists, relationships, validation rules, flows, approval process and all screenshots
2	Phase 2 documentation brief (Create Phase 2 Project Documentation PDF), provided by the project team	Structure and content requirements of this report
3	Salesforce Developer Edition sign-up page, https://developer.salesforce.com/signup (cited in Phase 1, Milestone 1)	Org creation procedure in Section 3

No other external sources were consulted. General Salesforce terms in this report follow the procedures and terminology of the Phase 1 document. Proposed rules and refinements are the team’s design suggestions and should be checked in the org before use.